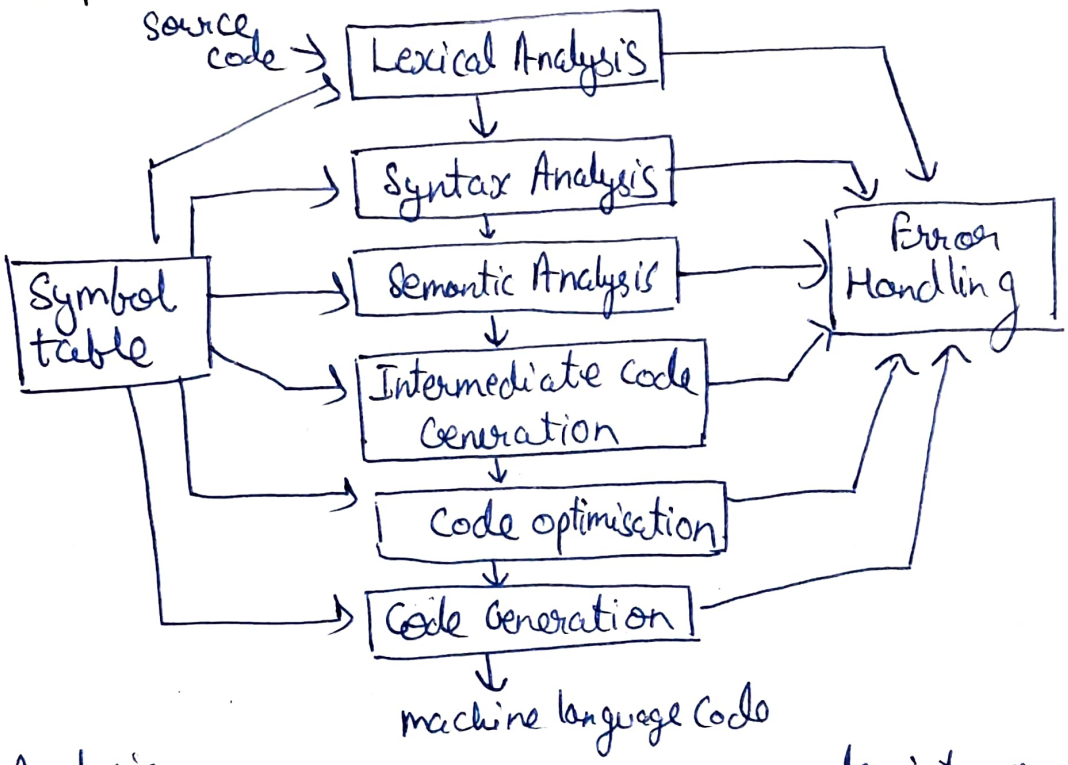
Name :- Achintya Bist
Class Roll no :- 4
Section : A
Semester : 6
Course : Compiler design.

Q1. Analyze the phases of the compiler by explaining the function of each phase with a suitable example.



1. Lexical Analysis
- It is responsible for converting the raw source code into a sequence of tokens. A token is the smallest unit of meaningful data in programming language.
- The lexical analyzer scans the source code character by character, grouping these characters into meaningful units based on the language's syntax rules.
- These tokens can represent keywords, identifiers, constants, operators or punctuation marks.

2. Syntax Analysis Parsing
- This phase ensures that the code flows the correct grammatical rules of the programming language.
- Verifies the sequence of tokens produced by the lexical analyzer is arranged in a valid way according to the language's syntax.

Name : Achintya Bisl
Section : A
Roll no : 4

- It checks whether the code adheres to the language's rules, such as correct use of operators, keywords and parenthesis.
- If the source code is not structured properly it will generate errors.
- Syntax Analyzer Uses Parse Tree and Syntax Tree to represent Structure of source code.

3. Semantic Analysis
   - This phase ensures the source code makes sense logically.
   - Checks whether the program has any semantic errors, such as type mismatches or undeclared variables.
   - Checks the meaning of the program by validating that the operations performed in that code are logically correct.
   - Ensures that the source code follows the rules of the programming language in terms of logic and data usage.

4. Intermediate code generation.
   - Intermediate code is a form of code that lies between the high-level source code and the final machine code.
   - It is not specific to any particular machine, making it portable and easier to optimize.
   - Acts as a bridge, simplifying the process of converting of source code into executable code.

eg : $a = b + c * d$

$t1 = c * d$

$t2 = b + t1$

$a = t2$

5. Code optimization
   - Process of improving the intermediate or target code to make the program run faster, use less memory or be more efficient without altering its functionality.
   - Involves techniques like removing ~~unnecess~~ unnecessary computations reducing redundancy, and reorganizing code to achieve better performance.

Name :- Ashutosh Bist
Class Roll no :- 4
Section :- A

eg. before optimization

```
for(int j=0; j<n; j++){
    x+y=z;
    a[j]=6*j;
}
```

after optimization

```
x+y=z;
for(int j=0; j<n; j++){
    a[j]=6*j;
}
```

6. Code Generation

- It is final phase of compiler, where the intermediate representation of the source program is translated into machine code or assembly code.
- The source code written in higher-level-language is transformed into a lower-level language which should have following minimum properties.
  - It should carry the exact meaning of the source code
  - It should be different in terms of CPU usage and memory management.

eg :

```
t1 = c*d
t2 = b+t1
a = t2
```

```
Load R1, c;
Load R2, d;
MUL R1, R2;
Load R3, b;
ADD R3, R1;
STORE a, R3;
```

Symbol table : Data structure being used and maintained by the compiler, consisting of all the identifier's name along with their types. It helps the compiler to function smoothly by finding the identifier's quickly.

2. Explain the cousins of the compiler and evaluate the roles of the preprocessor, assembler, linker and loader in achieving efficient program translation and execution.

A compiler is a system program that translates high-level language code into machine code. In the complete process of program translation and execution, the works along with several related system programs known as the cousins of the compiler.

Name :- Achintyo Bist
Section :- A
Roll no :- 4

These includes the preprocessor, assembler, linker and loader. Each plays a crucial role in ensuring correct, efficient and executable programs.

(i) Pre processor

The Preprocessor executes before the compilation phase, It prepares the source code by having directives and macros.

Functions:

a) Expands macro definitions (#define)
b) Includes header files (# include)
c) Handles conditional compilation (# ifdef, # endif)
d) Removes comments.

* By resolving macros and header inclusions early, the preprocessor simplifies the code for the compiler, reducing compilation complexity and enabling reusable codes through libraries.

(ii) Compiler:

The compiler translates the preprocessed source code into assembly or machine-level intermediate code.

Functions:

a) Performs lexical, syntax and semantic analysis
b) optimizes code
c) operates intermediate or target codes

The compiler ensures correctness, detects errors and applies optimizations for better runtime performance.

(iii) Assembler

The assembler converts assembly language code into machine-level object code.

Functions:

a) Translates mnemonics into binary instructions.
b) generates object files (.obj or .o)
c) Assigns memory addresses to instructions and variables

- Assembler produces low-level codes that closely maps to hardware ensuring fast execution and accurate memory usage.

Name : Achintya Bist
Section : A
Roll no. : 4

Many modern languages offer both ~~compilers~~ compilers and interpreters, or use a hybrid approach (such as compiling to bytecode which is the interpreted by a virtual machine).

1. C : compiler based
2. C++ : compiler based
3. Java : Hybrid
4. Python : Interpreter based
5. Javascript : Interpreter-based
6. Go (Golang) : compiler-based
7. Ruby : ~~Bad~~ Interpreter-based
8. Swift : Compiler based
9. PHP : Interpreter-based
10. Rust : compiler-based

4. Discuss the concepts of Native compiler, cross compiler and self hosting compiler highlighting their working principle and real-world use cases.

- Native compiler.

A native compiler is a compiler that runs on a specific platform and generates machine code that can only execute on the same platform, The host and target architectures are identical.

- Working principle : The compiler uses system libraries and API's specific to the host operating system and architecture and to to produce an executable file that runs directly on that environment.

eg :- operating system compilers : The GNU compiler and LLVM are frequently used as native compiler.

Name: Arkadyo Bist
Section: A
Roll no. 4

Name: Arkadyo Bist
Section: A

Cross compiler

A. cross compiler is a compiler that runs on one platform but generates machine code designed to run on a different platform. The compiler is specifically configured with target triples that specify the desired target architecture and operating system. It generates the object code compatible with the target environment.

eg:- Compiling applications for ios on mac OS machine is often considered a form of cross-compilation because the code generated for the ios device.

Self hosting

A ~~self~~ self hosting compiler is a compiler written in the same programming language that it compiles.

The key principle is bootstraping. The first version of the compiler must be compiled by a pre-existing compiler. Once the first version is running, subsequent versions of the compiler are compiled by previous versions of itself.

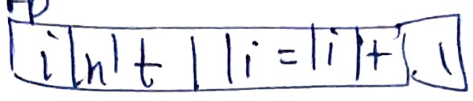
eg: It demonstrates the maturity and and completness of a programing language, proving the language is robust enough to manage its own complex implementation.

5 Explain the concept of input buffering in lexical analysis and analyze the two-buffer technique is more efficient.

Input buffering is a technique where the compiler reads input in blocks into a buffer instead of character by character from secondary storage. The lexical analyzer then processes characters from this buffer, which reduces no. of system calls.

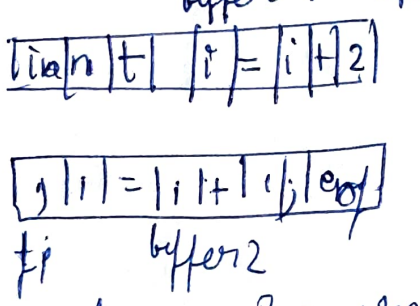
Name : Achintya Bist
Section : A
Roll no: 4

1. One buffer scheme



- In this scheme a single buffer is used to store a block of input. If a lexeme is longer than buffer size part of of lexeme may cross buffer boundary.

2. Two buffer scheme



Two buffer of equal size. They are filled alternatively. When the fp reaches the end of one buffer, the other buffer is refilled with the the next block of input.

Advantages: a) Reduced system calls
b) Improved performance
c) simpler design

Disadvantages: a) memory overhead
b) management complexity
c) lexeme boundary issue